

Raktárkezelő

Webalapú készletnyilvántartó és raktárkezelő rendszer

FEJLESZTŐI ÉS FELHASZNÁLÓI DOKUMENTÁCIÓ

Készítette:

Dobrocsi Zsolt

Tempfli Vivien

Tartalomjegyzék

Tartalomjegyzék.....	2
1. Bevezetés.....	4
1.1 A projekt célja és háttere.....	4
1.2 A dokumentáció felépítése és célja.....	5
2. Felhasznált technológiák.....	5
2.1 Áttekintés.....	5
2.2 Backend és adatelérés.....	6
2.3 Hitelesítés.....	6
2.4 API-dokumentáció fejlesztői környezetben.....	6
2.5 Frontend.....	7
3. Rendszerarchitektúra és projektstruktúra.....	7
3.1 A rendszer logikai felépítése.....	7
3.2 A kliens–szerver kommunikáció folyamata.....	7
3.3 Projektstruktúra.....	8
4. Adatbázis és adatmodell.....	8
4.1 A goods (termékek) tábla.....	9
4.2 A users (felhasználók) tábla.....	9
4.3 A history (készletmozgások) tábla.....	10
4.4 Kapcsolatok és törlési szabályok.....	10
4.5 Készletmozgás-kódok.....	11
5. Backend megvalósítás.....	11
5.1 Konfiguráció.....	11
5.2 Hitelesítés és jogosultságkezelés.....	12
5.3 Adatátviteli objektumok (DTO-k).....	12
5.4 Egységes API-válaszformátum és hibakezelés.....	13
5.5 Fontosabb üzleti szabályok.....	13
6. API-végpontok.....	14
6.1. /user végpontcsoport (UserController).....	14
6.2. /goods végpontcsoport (GoodsController).....	15
6.3. /history végpont (HistoryController).....	15
7. Frontend megvalósítás.....	17
7.1. Fájlstruktúra és felépítés.....	17
7.2. A közös api.js réteg.....	17
7.3. Szerepkör szerinti felület.....	18
7.4. Oldalankénti JavaScript-modulok.....	18
7.5. Hibakezelés és visszajelzés a felhasználó felé.....	19

8. Felhasználói dokumentáció.....	19
8.1. Bejelentkezés és az első felhasználó létrehozása.....	19
8.2. Termékek megtekintése, keresése és szűrése.....	20
8.3. Termékmezők jelentése.....	21
8.4. Termék felvitele, módosítása, törlése.....	21
8.5. Készletmozgás rögzítése.....	22
8.6. Készlettörténet megtekintése.....	24
9. Raktárvezetői (adminisztrációs) dokumentáció.....	24
9.1. Felhasználók listázása és felvitele.....	24
9.2. Felhasználói adatok és jelszó módosítása.....	25
9.3. Az utolsó Raktárvezető védelme.....	25
9.4. Új felhasználó felvitele.....	25
9.5. Felhasználó törlése.....	26
10. Telepítés és üzembe helyezés.....	27
10.1. Előfeltételek.....	27
10.2. Indítási sorrend.....	27
10.3. Gyakori induláskori hibák.....	27
11. Ismert korlátok, továbbfejlesztési lehetőségek.....	28
11.1. Jelenlegi korlátok.....	28
11.2. Biztonsági és üzemeltetési továbbfejlesztési irányok.....	28
11.3. Funkcionális és technikai továbbfejlesztési lehetőségek.....	29
12. Összefoglalás.....	29
13. Forrásjegyzék.....	30

1. Bevezetés

1.1 A projekt célja és háttere

A **Raktárkezelő** egy háromrétegű, webalapú alkalmazás, amely egy kisebb vállalkozás raktári készletgazdálkodását támogatja: a termékek nyilvántartását, a készletmozgások (bevételezés, kiadás) rögzítését és a korábbi mozgások visszakereshetőségét. A projekt egy szoftverfejlesztő tanfolyam csoportos vizsgamunkájaként készült, ezért két szempont vezérelte a fejlesztést: a funkciók valóban működjenek éles adatokon is, ugyanakkor a kód felépítése átlátható és tanulható maradjon.

Az alkalmazás klasszikus háromrétegű felépítést követ. Az adatokat egy **MySQL adatbázis** tárolja, az üzleti logikát és az adathozzáférést egy **ASP.NET Core Web API** valósítja meg, a megjelenítést pedig egy önálló, keretrendszer nélküli **HTML/CSS/JavaScript** frontend adja. A két oldal kizárólag a REST API-n keresztül kommunikál egymással, JSON formátumban.

A rendszer négy jogosultsági szinttel dolgozik, amelyek a mindennapi raktári munkakörökhöz igazodnak: Raktárvezető, Raktáros, Kereskedő és Anyagbeszerző. A jogosultságok érvényesítése nem csak a felületen, hanem minden esetben a szerveroldalon is megtörténik, így a menüpontok elrejtése a kliensen kényelmi funkció, a valódi védelmet a backend adja.

Szerepkör	Jogosultságok	Korlátozások
Raktárvezető (UserRank = 1)	Teljes hozzáférés: felhasználók és termékek kezelése, készletmozgás rögzítése, árak módosítása, minden lekérdezés.	Nincs korlátozás; ő az egyetlen szerepkör, amely felhasználót hozhat létre, módosíthat vagy törölhet.
Raktáros (UserRank = 2)	Termékek felvitele, módosítása és törlése, készletmozgások (bevételezés, kiadás) rögzítése.	Nem érheti el a felhasználókezelést, nem módosíthatja más felhasználó adatait.
Kereskedő (UserRank = 3)	Termékek és készletadatok megtekintése, kereskedelmi ár (eladási ár) módosítása.	Nem módosíthatja a termék többi adatát, nem rögzíthet készletmozgást, nem kezelhet felhasználókat.
Anyagbeszerző (UserRank = 4)	Készletmennyiségek és a minimumkészlet alapján a rendelendő mennyiség megtekintése.	Nem módosíthat és nem törölhet semmit, nem rögzíthet készletmozgást.

1. táblázat - A rendszer jogosultsági szintjei

A rendszerben addig nincs egyetlen felhasználó sem, amíg valaki létre nem hozza az elsőt a kezdőoldalon található regisztrációs űrlapon keresztül. Az így létrejövő első fiók automatikusan Raktárvezető jogosultságot kap; minden további felhasználót már egy belépett Raktárvezetőnek kell felvennie.

1.2 A dokumentáció felépítése és célja

A dokumentáció egyetlen, összefüggő anyagként tárgyalja a fejlesztői, a felhasználói és a tesztelési szempontokat, mert a projekt mérete ezt nem indokolja szétválasztani, és így könnyebben követhető, hogy egy adott funkció a rendszer mely rétegében és hogyan jelenik meg.

A dokumentáció felépítése a fejlesztés logikai menetét követi: elsőként a felhasznált technológiákat és a rendszer architektúráját mutatja be, majd az adatbázis modelljét, ezt követően a backend és a frontend megvalósítás részleteit fejlesztői szemszögből, majd a végfelhasználói és az adminisztrátori (Raktárvezetői) kézikönyvet, végül a telepítés lépéseit és a tesztelés eredményeit.

2. Felhasznált technológiák

2.1 Áttekintés

Ez a fejezet röviden áttekinti azokat a technológiákat, amelyekre az alkalmazás épül, és megmutatja, hogy a projektben pontosan milyen szerepet töltenek be. A technológiák önmagukban jól dokumentáltak, ezért a fejezet a részletes bemutatásuk helyett arra összpontosít, hogy miért és hol kerülnek felhasználásra a Raktárkezelőben.

Technológia	Szerepe a projektben
MySQL	Relációs adatbázis-kezelő rendszer; a goods, users és history táblák és a köztük lévő idegenkulcs-kapcsolatok tárolására szolgál.
ASP.NET Core Web API (.NET 10, C#)	A backend futtatókörnyezete és keretrendszere; a REST végpontokat, az üzleti logikát és a hitelesítést valósítja meg.
Entity Framework Core	Objektum-relációs leképező (ORM); a WarehouseContext osztályon keresztül biztosítja az adatbázis-hozzáférést LINQ lekérdezésekkel, kód gényál SQL helyett.
JWT (JSON Web Token) hitelesítés	A bejelentkezett felhasználó azonosítására és jogosultsági szintjének (szerepkörének) továbbítására szolgál a kliens és a szerver között, állapotmentes módon.
ASP.NET Core Identity PasswordHasher<T>	A jelszavak biztonságos, visszafejthetetlen tárolására (hashelésére) és ellenőrzésére használt beépített komponens.
Scalar (OpenAPI felület)	Fejlesztői eszköz, amely a futó backend API-végpontjait interaktív, böngészőből tesztelhető formában jeleníti meg fejlesztői környezetben.
HTML5 / CSS3 / JavaScript	A frontend alapja; keretrendszer nélkül, natív JavaScriptből (fetch API-val) épített felhasználói felület.
Bootstrap 5 és Bootstrap Icons	Reszponzív rácsrendszer, előre elkészített UI-komponensek (modális ablakok, gombok, űrlapok) és ikonkészlet a frontend gyors és egységes kialakításához.

2. táblázat - Felhasznált technológiák és szerepük

2.2 Backend és adatelérés

ASP.NET Core és Entity Framework Core

A backend ASP.NET Core Web API-ként épül fel: ez a keretrendszer adja a HTTP-kiszolgálót, a kérések útvonalválasztását (routing) a vezérlők (Controller) felé, valamint a beépített függőséginjektálást, amelyen keresztül a WarehouseContext adatbázis-kontextus minden vezérlőbe automatikusan bekerül. Az Entity Framework Core ORM-ként azt teszi lehetővé, hogy az adatbázis-táblákat (goods, users, history) közös C# osztályokként (entitásokként) kezeljük, és a lekérdezéseket LINQ-kifejezésekkel írjuk meg SQL helyett — az EF Core fordítja ezeket a háttérben tényleges SQL-utasításokká.

A projekt a MySql.EntityFrameworkCore csomagot használja szolgáltatóként, mert az adatbázis MySQL alapú. Az adatbázis-séma létrehozása fejlesztői kényelemből a Database.EnsureCreated() hívással történik induláskor a WarehouseContext-en keresztül, ami a projekt jelenlegi méreténél gyors és egyszerű megoldás; ennek korlátait a 12. fejezet tárgyalja.

2.3 Hitelesítés

JWT és ASP.NET Core Identity

A felhasználók azonosítására a projekt JSON Web Token (JWT) alapú, állapotmentes hitelesítést használ: bejelentkezéskor a szerver egy digitálisan aláírt tokent állít elő, amely a felhasználó azonosítóját és szerepkörét kódolja, és amelyet a kliens minden további kérésnél visszaküld. Ennek előnye, hogy a szervernek nem kell munkamenet-állapotot tárolnia — a token önmagában tartalmazza az ellenőrzéshez szükséges információt, és a lejáratí idejét is.

A jelszavak biztonságos tárolásához a projekt az ASP.NET Core Identity keretrendszer PasswordHasher<T> osztályát használja a teljes Identity-rendszer bevezetése nélkül; ez egy jól tesztelt, iparági szabványnak megfelelő egyirányú hashelő algoritmust biztosít, szózással (salt) együtt, így két azonos jelszó is eltérő hash-értéket kap.

2.4 API-dokumentáció fejlesztői környezetben

Scalar

Fejlesztés közben a backend a Scalar könyvtáron keresztül interaktív, OpenAPI-alapú felületet biztosít, amelyen az összes végpont böngészőből, hitelesítéssel együtt kipróbálható. Ez jelentősen megkönnyítette a fejlesztés és a tesztelés (11. fejezet) során az egyes végpontok elszigetelt, a frontendtől független ellenőrzését.

2.5 Frontend

HTML, BootStrap, CSS, natív JavaScript

A frontend tudatosan keretrendszer (React, Angular, Vue) nélkül készült: öt HTML-oldalból és a hozzájuk tartozó, natív JavaScript (ES6+) modulokból áll, amelyek a böngésző beépített fetch API-jával kommunikálnak a backenddel. Ez a választás a projekt méretéhez igazodik, és lehetővé tette, hogy a hangsúly a backend és az adatmodell helyes kialakításán legyen.

A vizuális megjelenést és a reszponzív elrendezést a Bootstrap 5 keretrendszer adja, kiegészítve a Bootstrap Icons ikonkészlettel; ezekre épül a projekt saját, kiegészítő CSS stíluslapja (css/style.css), amely a Bootstrap alapkomponeit hangolja a Raktárkezelő egységes megjelenéséhez.

3. Rendszerarchitektúra és projektstruktúra

3.1 A rendszer logikai felépítése

Az alkalmazás három, egymástól jól elkülönített rétegből áll:

- **Adatbázis** (MySQL): a goods, users és history táblákat tartalmazza, ezek szerkezetét a 4. fejezet mutatja be.
- **Backend** (ASP.NET Core Web API): kiszolgálja a HTTP-kéréseket, elvégzi a hitelesítést és a jogosultság-ellenőrzést, Entity Framework Core-on keresztül olvassa és írja az adatbázist, és JSON választ ad a kliensnek.
- **Frontend** (HTML/CSS/JavaScript): a böngészőben fut, a felhasználói interakciókból REST-kéréseket állít össze, és a kapott JSON adatokból építi fel a felületet.

A réteg közötti kommunikáció kizárólag a REST API-n keresztül történik: a frontend sosem éri el közvetlenül az adatbázist, minden adatműveletet a backend végez el, miután ellenőrizte a kérés jogosultságát.

3.2 A kliens–szerver kommunikáció folyamata

Egy tipikus felhasználói művelet (például egy termék módosítása) a következő lépéseken megy végig:

- A frontend JavaScript-kódja egy fetch hívást indít a megfelelő API-végpontra, a HTTP-fejlécben elhelyezve a bejelentkezéskor kapott JWT token.
- A backend a JWT middleware-en keresztül ellenőrzi a token érvényességét, majd a hozzá tartozó [Authorize] szabály alapján eldönti, hogy a felhasználó szerepköre jogosult-e a művelethez.

- Sikeres ellenőrzés esetén a vezérlő (Controller) végrehajtja az üzleti logikát, az Entity Framework Core-on keresztül olvassa vagy módosítja az adatbázist.
- A backend egységes formátumú JSON választ küld vissza (message és result mezőkkel), amelyet a frontend feldolgoz és megjelenít, vagy hiba esetén értesítésként kiír.

3.3 Projektstruktúra

A repository három fő mappára tagolódik, amelyek pontosan lekövetik a 3.1. alfejezetben bemutatott rétegeket:

```
Raktárkezelő/
├── Backend/WarehouseAPI/WarehouseAPI/
│   ├── Controllers/ (GoodsController, UserController, HistoryController)
│   ├── Models/ (Goods, User, History, WarehouseContext)
│   ├── Models/DTOs/ (adatátviteli objektumok)
│   ├── Program.cs (alkalmazás-összeállítás és -konfiguráció)
│   ├── appsettings.json / appsettings.Development.json
│   └── WarehouseAPI.csproj
├── Database/
│   └── create.sql (adatbázis- és tábla-létrehozó szkript)
└── Frontend/
    ├── index.html, products.html, stock.html, history.html, users.html
    ├── css/style.css
    └── js/ (api.js, login.js, products.js, stock.js, history.js, users.js)
```

A backend a szokásos ASP.NET Core Web API projektszerkezetet követi: a Controllers mappa a HTTP-végpontokat, a Models mappa az adatbázis-táblákat leképező entitásokat és a DTO-kat tartalmazza. A frontend öt önálló HTML-oldalból áll, amelyek egy közös JavaScript-réteget (api.js) használnak az API eléréséhez.

A táblázat nem tartalmazza a azon mappákat, amelyek nem képezik az alkalmazás szerves részét, így például a Dokumentációt, valamint a README fájlokat.

4. Adatbázis és adatmodell

Az adatbázis három táblából áll: a termékeket a goods, a felhasználókat a users, a készletmozgásokat pedig a history tábla tárolja. A táblák létrehozásához szükséges SQL-szkript a Database/create.sql fájlban található; ez automatikusan létrehozza a warehouse adatbázist és a táblákat, előre feltöltött adatok nélkül.

4.1 A goods (termékek) tábla

Mező	Típus	Leírás
idP	INT, elsődleges kulcs, automatikus	A termék egyedi azonosítója.
Article	VARCHAR(20)	Belső cikkszám, szabadon kialakítható termékazonosító.
Barcode	VARCHAR(13)	EAN-13 vonalkód.
Name	VARCHAR(100)	A termék megnevezése.
VAT	DECIMAL(5,2)	Alkalmazott áfakulcs.
LPPrice	DECIMAL(10,2)	Utolsó nettó beszerzési ár; bevételezéskor frissül.
SPrice	DECIMAL(10,2)	Nettó eladási ár.
Stock	FLOAT	Az aktuális készletmennyiség.
MinStock	FLOAT	A raktáron tartandó minimumkészlet.
Unit	VARCHAR(20)	Mennyiségi egység (pl. db, kg).
Shelf	VARCHAR(20)	Polckód, a termék fizikai helyének azonosítására.
Bundle	FLOAT	Gyűjtő (csomagolási) mennyiség.
BUnit	VARCHAR(20)	Gyűjtő egység.

3. táblázat - A goods tábla mezői

4.2 A users (felhasználók) tábla

Mező	Típus	Leírás
idU	INT, elsődleges kulcs, automatikus	A felhasználó egyedi azonosítója.
UserName	VARCHAR(100), egyedi (UX_users_UserName)	Bejelentkezési felhasználónév; nem lehet kétszer felvenni.
FullName	VARCHAR(100)	A felhasználó teljes neve, megjelenítéshez.
PasswordHash	VARCHAR(255), kötelező	A jelszó ASP.NET Core Identity PasswordHasher által előállított, hashelt formája — a nyers jelszó soha nem kerül tárolásra.
UserRank	INT	A szerepkör azonosítója (1–4, lásd 1. táblázat).

4. táblázat - A users tábla mezői

4.3 A history (készletmozgások) tábla

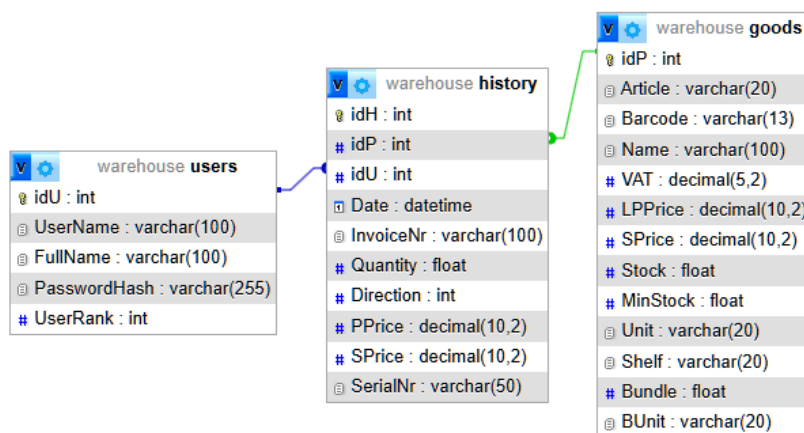
Mező	Típus	Leírás
idH	INT, elsődleges kulcs, automatikus	A készletmozgási bejegyzés egyedi azonosítója.
idP	INT, idegen kulcs → goods.idP	Az érintett termék; a termék törlésekor a hozzá tartozó mozgások is törlődnek (CASCADE).
idU	INT, idegen kulcs → users.idU	A mozgást rögzítő felhasználó; a felhasználó nem törölhető, amíg van hozzá kapcsolódó mozgás (RESTRICT).
Date	DATETIME	A mozgás rögzítésének időpontja (szerveridő).
InvoiceNr	VARCHAR(100)	Bizonylatszám; ez alapján kereshetők vissza az összetartozó mozgások.
Quantity	FLOAT	A mozgatott mennyiség.
Direction	INT	Készletmozgás-kód (lásd 3. táblázat).
PPrice	DECIMAL(10,2)	A mozgáshoz rögzített beszerzési ár.
SPrice	DECIMAL(10,2)	A mozgás időpontjában érvényes eladási ár.
SerialNr	VARCHAR(50)	Opcionális sorozatszám.

5. táblázat - A history tábla mezői

4.4 Kapcsolatok és törlési szabályok

A history tábla két idegenkulcs-kapcsolattal kötődik a másik két táblához:

- **goods** → **history** (CASCADE): ha egy termék törlődik, a hozzá tartozó összes készletmozgási bejegyzés is automatikusan törlődik. Az alkalmazás emellett üzleti szabályként azt is megköveteli, hogy egy termék csak 0 készlet esetén legyen törölhető (lásd 5.5. alfejezet).
- **users** → **history** (RESTRICT): egy felhasználó nem törölhető, amíg van hozzá kapcsolódó készletmozgási bejegyzés; egy ilyen törlési kísérletet az adatbázis idegenkulcs-megszorítása utasít el, amelyet a backend hibaként kezel le.



1. ábra - Az adatbáziskapcsolatok áttekintése tervező nézetben a phpMyAdmin felületén

4.5 Készletmozgás-kódok

A készletmozgásokat egy háromjegyű kód (Direction / Mcode) azonosítja. A 100-as tartomány a készletet növelő, a 200-as tartomány a készletet csökkentő mozgásokat jelöli; a beszerzési ár csak a 101-es és 102-es kódnál frissül.

Kód	Jelentés	Hatás a készletre
101	Beszerezés — a beszerzési ár (LPPrice) is frissül, ha meg van adva.	+ mennyiség
102	Bizományos bevételezés — a beszerzési ár is frissül, ha meg van adva.	+ mennyiség
103	Téves selejtezés visszavétele.	+ mennyiség
104	Saját rezszi anyag visszavétele.	+ mennyiség
201	Értékesítés.	– mennyiség
202	Bizományos kiadás.	– mennyiség
203	Selejtezés.	– mennyiség
204	Saját rezszi felhasználás.	– mennyiség

6. táblázat - készletmozgás-kódok

A kódok háromjegyű, olvasható felépítése (1xx / 2xx) megkönnyíti, hogy egy history-bejegyzés Direction mezőjéből önmagában, adatbázis-lekérdezés szintjén is eldönthető legyen, hogy növelő vagy csökkentő mozgásról van-e szó (pl. WHERE Direction < 200 az összes bevételezésre szűr). A konkrét szöveges elnevezéseket (pl. „Beszerezés”, „Értékesítés”) a frontend getMovementName() függvénye rendeli a kódokhoz megjelenítéskor; a backend maga csak a numerikus kódot tárolja és validálja.

5. Backend megvalósítás

5.1 Konfiguráció

Az alkalmazás konfigurációja az appsettings.json és a csak fejlesztői környezetben betöltődő appsettings.Development.json fájlokban található. Utóbbi tartalmazza a JWT aláíró kulcsát (Jwt:Key), hogy egy frissen klónozott repository külön titokbeállítás nélkül is elindítható legyen; éles környezetben ezt az értéket biztonságos módon (pl. környezeti változóból vagy titokkezelőből) kell megadni.

```
"ConnectionStrings": { "MySQL":
"server=localhost;database=warehouse;user=root;password=" }
"Jwt": { "Key": "...", "Issuer": "WarehouseAPI", "Audience":
"WarehouseFrontend", "ExpirationMinutes": 60 }
"Cors": { "Origins": [ "http://localhost:5500", "http://127.0.0.1:5500" ] }
```

A Program.cs induláskor ellenőrzi, hogy a Jwt:Key be van-e állítva és legalább 32 karakter hosszú-e; ha nem, az alkalmazás azonnal hibával leáll, elkerülve, hogy gyenge vagy hiányzó kulccsal fusson tovább. Az engedélyezett frontend-eredetek (CORS) szintén konfigurálhatók, alapértelmezetten a Live Server 5500-as portjára vannak beállítva.

5.2. Hitelesítés és jogosultságkezelés

A rendszer JWT Bearer hitelesítést használ. Bejelentkezéskor (POST /user/login) a backend ellenőrzi a felhasználónevet és a jelszót az ASP.NET Core Identity PasswordHasher<User> komponensével, majd sikeres ellenőrzés esetén egy tokent állít elő, amely a felhasználó azonosítóját, nevét és szerepkörét (UserRank) tartalmazza claim-ként.

A frontend minden további kérésnél ezt a tokent küldi el az Authorization: Bearer <token> fejlécben. A backend minden védett végpontnál — a Controllerre vagy az egyes metódusra helyezett [Authorize] attribútummal — ellenőrzi a token érvényességét és a felhasználó szerepkörét; az [Authorize(Roles = "1,2")] jelölés például azt jelenti, hogy a végpontot csak Raktárvezető vagy Raktáros hívhatja. Az [AllowAnonymous] attribútummal jelölt végpontok (regisztráció, bejelentkezés, felhasználó-létezés ellenőrzése) hitelesítés nélkül is elérhetők.

A jelszavak soha nem kerülnek egyszerű szöveggént tárolásra: a PasswordHasher egyirányú hash-t képez, amelyet a rendszer bejelentkezéskor és jelszóváltoztatáskor is csak összehasonlításra használ, visszafejtésre nem.

Egy sikeres bejelentkezési kérés és a rá adott válasz felépítése a következő:

```
POST /user/login
Content-Type: application/json

{
  "userName": "raktarvezeto1",
  "password": "*****"
}
200 OK
{
  "message": "Sikeres bejelentkezés.",
  "result": {
    "token": "eyJhbGciOiJIUzI1NiIs... ",
    "user": { "idU": 1, "userName": "raktarvezeto1", "fullName": "Kovács Anna", "userRank": 1 }
  }
}
```

5.3. Adatátviteli objektumok (DTO-k)

A backend a belső entitások (Goods, User, History) helyett — a felhasználói jelszóhash véletlen kiszivárgásának elkerülése és a bemenet szűkítése érdekében — dedikált DTO osztályokat használ a kérésekben és egyes válaszokban.

DTO	Használat	Fontosabb mezők
LoginDto	Bejelentkezés kérésben (POST /user/login).	UserName, Password
AddUserDto	Új felhasználó létrehozásakor (POST /user).	UserName*, FullName*, Password*, UserRank*
UpdateUserDto	Felhasználó adatainak módosításakor (PUT /user).	UserName*, FullName*, UserRank*
UserResponseDto	Minden felhasználó-lekérdezés válaszában — jelszóhash nélkül.	IdU, UserName, FullName, UserRank
ChangePasswordDto	Saját és adminisztrátori jelszócseréhez.	CurrentPassword, NewPassword, ConfirmPassword
AddGoodsDto	Új termék felvitelekor (POST /goods).	Name* és a leíró mezők (Article, Barcode, Vat, Sprice, MinStock, Unit, Shelf, Bundle, Bunit)
UpdateGoodsDto	Termék adatainak módosításakor (PUT /goods).	A leíró mezők, a Sprice-szal együtt
MovementGoodsDto	Készletmozgás rögzítésekor (PUT /goods/movement).	Mcode, Stock (mennyiség), Lpprice, InvoiceNr, SerialNr

6. táblázat - A projekt DTO osztályai

5.4. Egységes API-válaszformátum és hibakezelés

A vezérlők a válaszokat egységesen egy message (szöveges üzenet) és egy result (a létrehozott, módosított vagy lekérdezett adat) mezőt tartalmazó JSON objektumként adják vissza. A HTTP-státuszkódok is következetesek: 200/201 sikeres művelet esetén, 400 hibás kérés vagy üzleti szabály megsértése esetén, 401 hiányzó vagy érvénytelen hitelesítés esetén, 403 érvényes hitelesítés melletti jogosultsághiány esetén, 404 nem található erőforrás esetén, 409 pedig ütköző adat (pl. már foglalt felhasználónév) esetén.

Minden vezérlőmetódus try–catch blokkba van csomagolva: a váratlan kivételeket a szerver naplózza (Console.Error), a kliens felé pedig egy általános, magyar nyelvű hibaüzenettel tér vissza — így a technikai részletek nem szivárognak ki, a felhasználó mégis érthető visszajelzést kap.

5.5. Fontosabb üzleti szabályok

A validáció és az egyszerű adattárolás felett a backend a következő, a raktári folyamatokból adódó üzleti szabályokat kényszeríti ki:

- **Az utolsó Raktárvezető védelme:** egy felhasználó szerepköre nem módosítható másra, és egy felhasználó nem törölhető, ha ő az utolsó Raktárvezető a rendszerben — így az alkalmazás soha nem maradhat adminisztrátor nélkül.

- **Termék törlése csak 0 készlettel:** egy termék csak akkor törölhető, ha aktuális készlete pontosan 0; ellenkező esetben a rendszer elutasítja a kérést.
- **Készletmozgás validációja:** a mozgatott mennyiség csak pozitív szám lehet, a mozgáskód csak a nyolc érvényes kód egyike lehet, és egy csökkentő (200 fölötti kódú) mozgás nem eredményezhet negatív készletet.
- **Egyedi felhasználónév:** a users tábla adatbázis-szintű egyedi indexe (UX_users_UserName) és a vezérlő előzetes ellenőrzése együtt akadályozza meg a duplikált felhasználónevek felvételét.
- **Első felhasználó speciális kezelése:** amíg a users tábla üres, a POST /user végpont hitelesítés nélkül is elérhető, és az így létrejövő fiók automatikusan Raktárvezető szerepkört kap; ezután a végpont csak bejelentkezett Raktárvezető számára érhető el.

6. API-végpontok

Ez a fejezet a backend teljes, ténylegesen implementált végpontlistáját mutatja be vezérlőnként csoportosítva. A „Jogosultság” oszlopban a „Nyilvános” azt jelenti, hogy a végpont [AllowAnonymous] jelöléssel hitelesítés nélkül is elérhető; a „Bejelentkezett” azt, hogy bármelyik érvényes tokennel rendelkező felhasználó elérheti, szerepkörtől függetlenül.

6.1. /user végpontcsoport (UserController)

Metódus	Végpont	Jogosultság	Leírás
POST	/user	Nyilvános*	Felhasználó létrehozása. Az első felhasználó bárki által, Raktárvezetőként jön létre; utána csak Raktárvezető hozhat létre újabbat, a kért szerepkörrel.
GET	/user	Raktárvezető	Az összes felhasználó lekérdezése (UserResponseDto lista, jelszóhash nélkül).
GET	/user/names	Bejelentkezett	Felhasználói azonosító–teljes név párok lekérdezése (pl. megjelenítéshez).
GET	/user/exists	Nyilvános	Megmondja, van-e már felhasználó a rendszerben — ez dönti el, hogy a kezdőlap a regisztrációs vagy a bejelentkezési űrlapot mutatja.
GET	/user/byid?id=	Bejelentkezett	Egy adott felhasználó adatainak lekérdezése.
PUT	/user?id=	Raktárvezető	Felhasználó adatainak (név, teljes név, szerepkör) módosítása; védi az utolsó Raktárvezetőt.

Metódus	Végpont	Jogosultság	Leírás
DELETE	/user?id=	Raktárvezető	Felhasználó törlése; védi az utolsó Raktárvezetőt, és elutasítja a törlést, ha a felhasználóhoz még kapcsolódik készletmozgás.
POST	/user/login	Nyilvános	Bejelentkezés felhasználónévvel és jelszóval; sikeres esetben JWT token és a felhasználó adatait adja vissza.
PUT	/user/change-password	Bejelentkezett	Saját jelszó módosítása; a jelenlegi jelszó ellenőrzése kötelező.
PUT	/user/change-password-manager?id=	Raktárvezető	Másik felhasználó jelszavának módosítása, a jelenlegi jelszó ismerete nélkül.

7. táblázat - Felhasználókezelő végpontok

* A POST /user speciális eset: amíg nincs felhasználó a rendszerben, hitelesítés nélkül is hívható; ha már van legalább egy, csak bejelentkezett Raktárvezető használhatja (lásd 5.5. alfejezet).

6.2. /goods végpontcsoport (GoodsController)

Metódus	Végpont	Jogosultság	Leírás
POST	/goods	Raktárvezető, Raktáros	Új termék felvitele; a készlet és a beszerzési ár kezdőértéke 0.
GET	/goods	Bejelentkezett	Az összes termék lekérdezése.
GET	/goods/byid?id=	Bejelentkezett	Egy adott termék adatainak lekérdezése.
PUT	/goods?id=	Raktárvezető, Raktáros	Termék leíró adatainak (és eladási árának) módosítása.
PUT	/goods/price?id=	Raktárvezető, Kereskedő	Kizárólag az eladási ár módosítása.
DELETE	/goods?id=	Raktárvezető, Raktáros	Termék törlése; csak 0 készlet esetén engedélyezett.
GET	/goods/getAllGoodsHistory?id=	Bejelentkezett	Egy termék adatai a hozzá tartozó összes készletmozgással együtt.
PUT	/goods/movement?id=	Raktárvezető, Raktáros	Készletmozgás rögzítése (bevételezés vagy kiadás); frissíti a készletet, szükség esetén a beszerzési árat, és új history bejegyzést hoz létre.

8. táblázat - Termék- és készletkezelő végpontok

6.3. /history végpont (HistoryController)

Metódus	Végpont	Jogosultság	Leírás
GET	/history	Bejelentkezett	Az összes rögzített készletmozgás lekérdezése.

9. táblázat - Előzménylekérdező végpont

A készletmozgás rögzítésének (PUT /goods/movement) request törzse a MovementGoodsDto-t követi: a Mcode a mozgás típusát (lásd 5. táblázat), a Stock a mozgatott mennyiséget, a Lpprice az esetleges új beszerzési árat, az InvoiceNr és a SerialNr pedig a bizonylat-, illetve sorozatszámot adja meg. A végpont elutasítja a kérést, ha a mennyiség nem pozitív, ha a kód nem szerepel az érvényes kódok között, vagy ha egy csökkentő mozgás a rendelkezésre álló készletnél nagyobb mennyiséget venne ki.

```
PUT /goods/movement?id=17
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "mcode": 101,
  "stock": 50,
  "lpprice": 1200,
  "invoiceNr": "SZ-2026-00042",
  "serialNr": ""
}
200 OK
{
  "message": "A készletmozgás sikeresen rögzítve.",
  "result": { "idP": 17, "stock": 174, "lpprice": 1200 }
}
```

Ha a mozgás típusa csökkentő (200 fölötti kód), és a kért mennyiség meghaladja a rendelkezésre álló készletet, a végpont a következőhöz hasonló hibaválaszt adja, és a készlet nem változik:

```
400 Bad Request
{
  "message": "Nincs elegendő készlet a mozgás végrehajtásához."
}
```

Egy jogosultság nélküli hívás (pl. Kereskedő szerepkörrel meghívott PUT /goods/movement) a backend [Authorize(Roles="1,2")] ellenőrzésén akad el, még mielőtt bármilyen üzleti logika lefutna:

```
403 Forbidden
```

7. Frontend megvalósítás

7.1. Fájlstruktúra és felépítés

A frontend öt önálló HTML-oldalból, egy közös stíluslapból (css/style.css) és hat JavaScript-fájlból áll. Minden oldal a Bootstrap 5 keretrendszerre és a Bootstrap Icons ikonkészletre épül, kliensoldali JavaScript keretrendszer (React, Vue stb.) nélkül. A frontend statikus fájlokból áll, ezért fejlesztés közben egy egyszerű statikus kiszolgálóval (pl. VS Code Live Server) futtatható.

Oldal	Funkció	Ki éri el
index.html	Bejelentkezés / első felhasználó regisztrációja, majd a főoldali irányítópult (kártyák a modulokhoz, saját jelszó módosítása, kijelentkezés).	Mindenki (a modulok kártyák szerepkör szerint jelennek meg).
products.html	Termékek listázása, keresése, szűrése (készlet szerint) és rendezése; termék felvitel, módosítása, törlése.	Megtekintés: mindenki. Felvitel/törlés: 1–2. Szerkesztés: 1–3 (Kereskedőnél csak az ár).
stock.html	Készletmozgás rögzítése (termék, mozgástípus, mennyiség, ár, bizonylatszám); bizonylatszám szerinti visszakeresés.	Raktárvezető, Raktáros (1–2).
history.html	A rögzített készletmozgások listázása, keresése és rendezése.	Mindenki, aki be van jelentkezve.
users.html	Felhasználók listázása, létrehozása, adatainak módosítása, jelszavának módosítása, törlése.	Kizárólag Raktárvezető (1).

10. táblázat - A frontend oldalai

7.2. A közös api.js réteg

Az api.js fájl minden oldal számára elérhető, és két fő feladatot lát el: egységesíti a backend hívását, és tartalmazza az újrafelhasznált UI-segédfüggvényeket (betöltés-jelző, hibaüzenet-sáv, gombok letiltása mentés közben, értesítési buborékok).

Az authFetch(url, options) függvény minden védett kéréshez hozzáadja az Authorization: Bearer fejléct a böngésző localStorage-ában tárolt tokenből, és — ha a válasz 401 Unauthorized — automatikusan törli a bejelentkezési adatokat, és visszairányítja a felhasználót a kezdőoldalra. Ennek köszönhetően a lejárt vagy érvénytelen token kezelése egyetlen helyen van megvalósítva, nem oldalanként.

A hasRole(...szerepkörök) segédfüggvény a bejelentkezett felhasználó UserRank értékét veti össze a megengedett szerepkörökkel, és ez alapján rejt el vagy jelenít meg gombokat és menüpontokat (pl. „Új termék” gomb csak Raktárvezetőnek és Raktárosnak). Fontos, hogy ez

kizárólag a felület kényelmét szolgálja — a tényleges jogosultság-ellenőrzést minden esetben a backend végzi el, a felület esetleges megkerülése esetén is.

A két segédfüggvény egyszerűsített felépítése:

```
async function authFetch(url, options = {}) {
  const token = localStorage.getItem("token");
  const headers = { ...options.headers, Authorization: `Bearer ${token}` };
  const response = await fetch(url, { ...options, headers });
  if (response.status === 401) {
    localStorage.clear();
    window.location.href = "index.html";
  }
  return response;
}

function hasRole(...allowedRanks) {
  const user = getLoggedInUser();
  return !!user && allowedRanks.includes(user.userRank);
}
```

7.3. Szerepkör szerinti felület

A products.js jól szemlélteti, hogyan igazodik a felület a szerepkörökhöz anélkül, hogy külön oldalt kellene készíteni: a szerkesztő ablak minden, terméklistát látó felhasználónak (1–3. szerepkör) megnyílik, de Kereskedő (3) esetén a rendszer az összes mezőt letiltja, kivéve az eladási árat, és mentéskor nem az általános termékmódosító, hanem a kifejezetten erre a célra szolgáló PUT /goods/price végpontot hívja meg. Ezzel a felület és a backend jogosultsági logikája következetesen ugyanazt az üzleti szabályt tükrözi.

7.4. Oldalankénti JavaScript-modulok

A közös api.js rétegen felül minden HTML-oldalhoz tartozik egy saját, csak az adott oldal logikáját tartalmazó JavaScript-fájl. Ez a felosztás jól mutatja a felelőségek szétválasztását, és megkönnyíti, hogy egy-egy modult a többitől függetlenül lehessen módosítani vagy tesztelni.

- **products.js:** betölti és megjeleníti a terméklistát, kezeli a keresést, a szűrést és a rendezést, valamint a termék felvitelére, módosítására és törlésére szolgáló ablakokat a 7.3. alfejezetben bemutatott szerepkör-függő viselkedéssel.
- **stock.js:** felépíti a készletmozgás-rögzítő űrlapot, a legördülő menüket a termékekkel és a nyolc mozgástípussal (5. táblázat) tölti fel, elküldi a PUT /goods/movement kérést, és megjeleníti a bizonylatszám szerinti korábbi mozgásokat.
- **history.js:** lekéri és táblázatos formában jeleníti meg a GET /history végpont teljes eredményét, kliensoldali kereséssel és rendezéssel kiegészítve.

- **users.js:** kizárólag a Felhasználók oldalon töltődik be; a felhasználók listázását, létrehozását, módosítását, jelszócseréjét és törlését vezérli, a 9. fejezetben leírt Raktárvezetői műveleteknek megfelelően.
- **login.js:** az index.html oldalon fut; eldönti a GET /user/exists válasza alapján, hogy a regisztrációs vagy a bejelentkezési űrlapot mutassa, kezeli a bejelentkezést, és felépíti az irányítópult szerepkör szerinti modulkátyáit.

7.5. Hibakezelés és visszajelzés a felhasználó felé

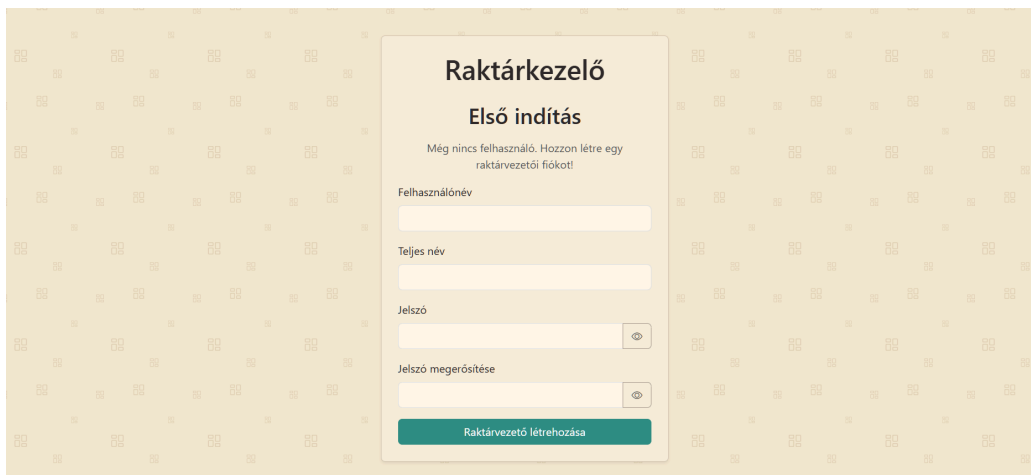
Minden API-hívás try-catch blokkban fut; hiba esetén a felület a showNotification() függvénnyel egy eltűnő értesítési sávot jelenít meg a backend által küldött, magyar nyelvű üzenettel. A táblázatok betöltés közben helykitöltő (skeleton) sorokat mutatnak, hibás betöltés esetén pedig egy „Újrapróbálás” gombot, hogy a felhasználó ne akadjon el egy átmeneti hálózati hiba miatt.

8. Felhasználói dokumentáció

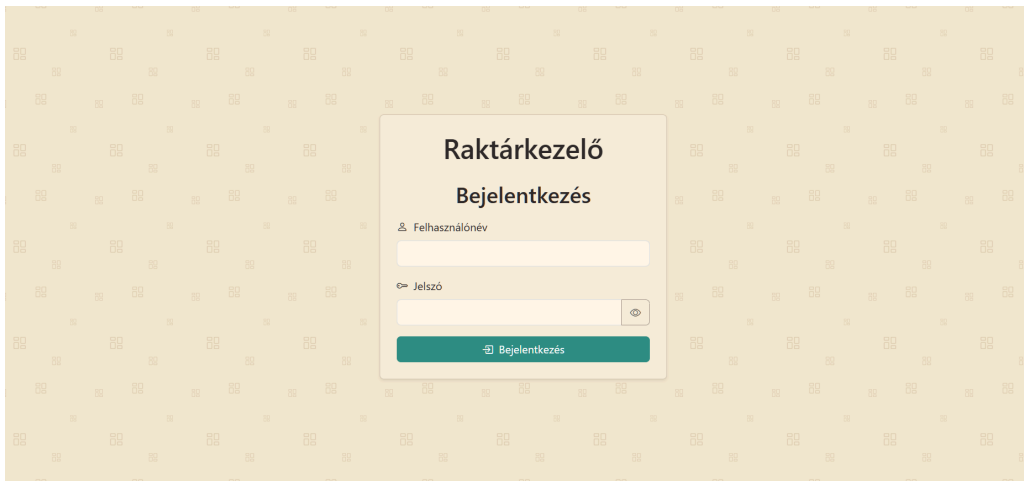
Ez a fejezet a Raktárkezelő mindennapi használatát mutatja be a Raktáros, a Kereskedő és az Anyagbeszerző szemszögéből. A Raktárvezetőre vonatkozó, felhasználókezeléssel kapcsolatos lépéseket a 9. fejezet tárgyalja külön.

8.1. Bejelentkezés és az első felhasználó létrehozása

Az alkalmazás megnyitásakor a rendszer automatikusan ellenőrzi, van-e már felhasználó az adatbázisban. Ha nincs, regisztrációs űrlap jelenik meg, ahol az első, Raktárvezetői jogú fiók hozható létre felhasználónév, teljes név és jelszó megadásával. Ha már létezik felhasználó, a szokásos bejelentkezési űrlap jelenik meg, ahol a felhasználónév és a jelszó megadása után a Bejelentkezés gombra kattintva érhető el a rendszer.



2. ábra - Első felhasználó létrehozása, mikor nincs felhasználó az adatbázisban



3. ábra - Szokásos bejelentkezési felület

Sikeres bejelentkezés után az irányítópulton megjelenik a bejelentkezett felhasználó neve és szerepköre, valamint a szerepkörnek megfelelő modulkárták (Termékek, Készletmozgás, Készlettörténet, Felhasználók).



4. ábra - Irányítópult

8.2. Termékek megtekintése, keresése és szűrése

A Termékek oldalon a raktár teljes termékkészlete táblázatos formában jelenik meg. A keresőmező a termék nevére, cikkszámára és vonalkódjára egyszerre keres, a szűrő legördülő menü a készletállapot szerint (pl. minimumkészlet alatti termékek) szűkíti a listát, a rendezés legördülő menü pedig különböző szempontok szerint (név, mennyiség stb.) rendezi a táblázatot. Azoknál a termékeknél, amelyeknél a rendelendő mennyiség pozitív — vagyis a készlet a minimumkészlet alá csökkent —, egy figyelmeztető ikon jelzi az utánrendelés szükségességét.

ID	Cikkszám	Vonalkód	Megnevezés	ÁFA	Beszerzési ár	Eladási ár	Készlet	Min. készlet	Rendelendő mennyiség	Egység	Polc	Gyűjtő	Műveletek
4	002	002	Üveggolyó	27%	25 Ft	100 Ft	80	12	0	db	Játék	6	zsák
3	001	001	Kis fa kocka	27%	100 Ft	300 Ft	32	4	0	db	Játék	4	doboz
5	003	003	Hintáló	27%	0 Ft	2400 Ft	0	1	1	db	Játék	1	doboz

5.ábra - Terméklista, eladási ár szerint rendezve

8.3. Termékmezők jelentése

Mező	Jelentés
Cikkszám / Vonalkód	A termék belső, illetve EAN-13 azonosítója; mindkettőre lehet keresni.
Megnevezés	A termék neve; az egyetlen kötelezően megadandó mező felvitelkor.
Készlet	Az aktuális raktárkészlet, amelyet kizárólag készletmozgás rögzítése módosít.
Minimumkészlet	Az a mennyiség, amely alá csökkenve a rendszer figyelmeztető ikonnal jelzi az utánrendelés szükségességét.
Beszerzési ár	Az utolsó rögzített nettó beszerzési ár; bevételezéskor (101, 102 kód) frissül.
Eladási ár	A nettó eladási ár; Raktárvezető és Raktáros a termékszerkesztésben, Kereskedő a saját, korlátozott szerkesztőjében módosíthatja.
Polc, Egység, Gyűjtő mennyiség	A raktári tárolást és a mennyiségi egységet (pl. doboz, karton) segítő kiegészítő adatok.

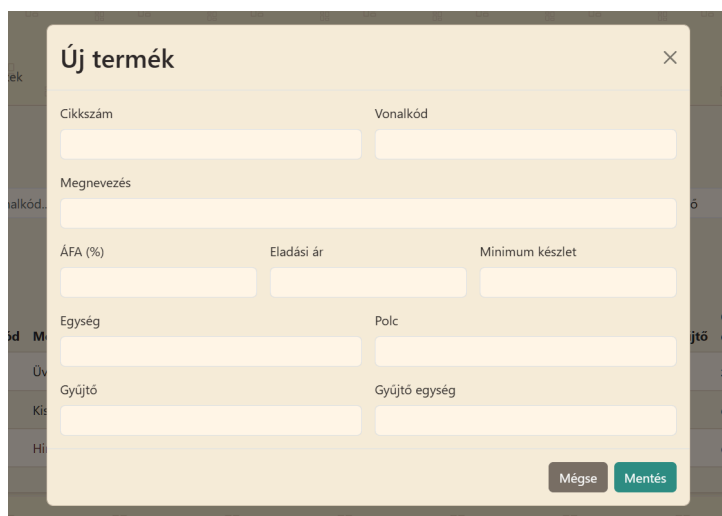
11.táblázat - A terméklistában és termékúrlapon megjelenő mezők

8.4. Termék felvitele, módosítása, törlése

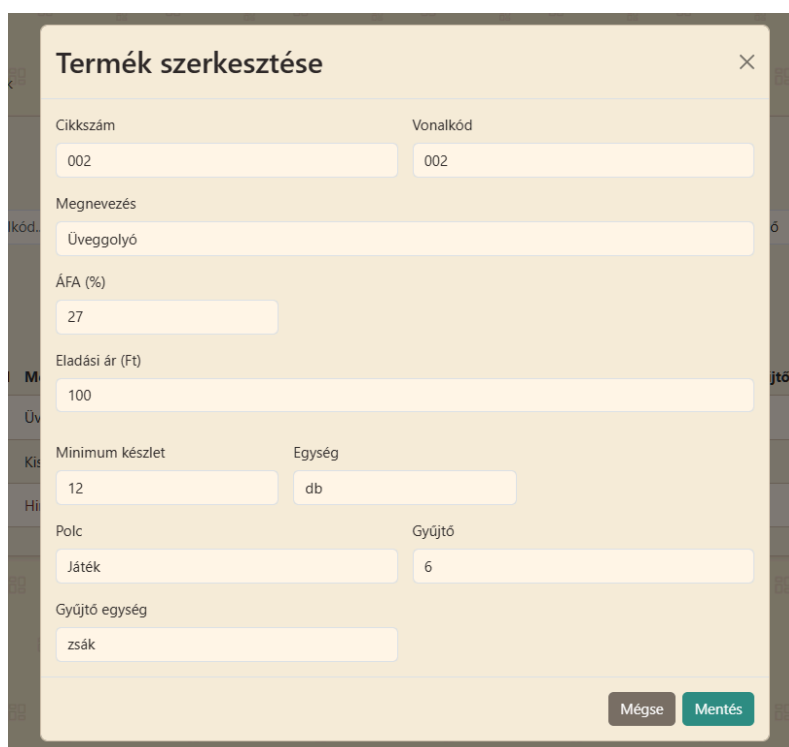
Raktárvezetőként vagy Raktárosként bejelentkezve az „Új termék” gombbal vehető fel termék: kötelező a megnevezés megadása, a további mezők (cikkszám, vonalkód, áfakulcs, eladási ár, minimumkészlet, mennyiségi egység, polckód, gyűjtő mennyiség) opcionálisak. Az újonnan felvitt termék kezdő készlete és beszerzési ára mindig 0, ezeket az első bevételezés tölti fel.

A szerkesztés ikonnal megnyitható ablakban a leíró adatok módosíthatók; Kereskedőként bejelentkezve ugyanez az ablak nyílik meg, de csak az eladási ár mező szerkeszthető, a többi mező írásvédett. A törlés ikon csak akkor jelenik meg (és csak akkor sikeres), ha a termék

készlete pontosan 0 — nem nulla készletű terméket a rendszer nem enged törölni, hogy a korábbi mozgások mindig visszakereshetők maradjanak egy létező termékhez kötve.

A screenshot of a web application window titled "Új termék" (New product). The window contains several input fields for product information: "Cikkszám" (Article number) and "Vonalkód" (Barcode) at the top; "Megnevezés" (Name) below them; "ÁFA (%)" (VAT %), "Eladási ár" (Selling price), and "Minimum készlet" (Minimum stock) in the next row; "Egység" (Unit) and "Polc" (Shelf) below that; and "Gyűjtő" (Collector) and "Gyűjtő egység" (Collector unit) at the bottom. At the bottom right of the form are two buttons: "Mégse" (Cancel) and "Mentés" (Save).

6.ábra - Új termék ablak

A screenshot of a web application window titled "Termék szerkesztése" (Edit product). The form contains the same fields as the "Új termék" window, but with pre-filled data: "Cikkszám" is 002, "Vonalkód" is 002, "Megnevezés" is Üveggolyó, "ÁFA (%)" is 27, "Eladási ár (Ft)" is 100, "Minimum készlet" is 12, "Egység" is db, "Polc" is Játék, "Gyűjtő" is 6, and "Gyűjtő egység" is zsák. At the bottom right are "Mégse" and "Mentés" buttons.

7.ábra - Termék szerkesztése ablak

8.5. Készletmozgás rögzítése

A Készletmozgás oldal (csak Raktárvezető és Raktáros számára elérhető) egy űrlapon rögzíti a bevételezéseket és a kiadásokat. A felhasználó kiválasztja a terméket, a mozgás típusát (lásd az 5. táblázatban felsorolt nyolc kódot), megadja a mennyiséget, szükség esetén az árat,

valamint a bizonylat- és sorozatszámot. Beszerzés (101) vagy bizományos bevételezés (102) esetén a megadott beszerzési ár frissíti a termék nyilvántartott beszerzési árát; kiadás típusú mozgásnál a rendszer ellenőrzi, hogy van-e elegendő készlet, és hibaüzenetet ad, ha nincs.

A bizonylatszám mezőbe gépelve az oldal alján automatikusan megjelenik az adott bizonylatszámhoz korábban rögzített mozgások listája, megkönnyítve egy-egy szállítólevél vagy számla tételeinek utólagos ellenőrzését.

Ehhez a bizonylathoz rögzített mozgások				
Kis fa kocka				
2026. 09. 13. 18:18:13	101 - Beszerzés	Mennyiség: 32	Beszerzési ár: 100 Ft	
Üveggolyó				
2026. 09. 13. 18:18:35	101 - Beszerzés	Mennyiség: 80	Beszerzési ár: 25 Ft	

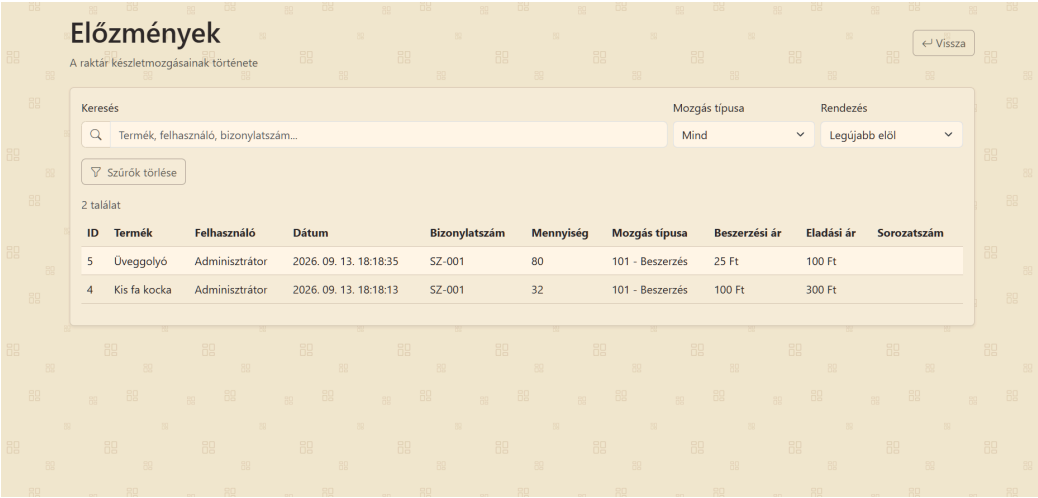
8. ábra - Készletmozgás rögzítő űrlap, bizonylat szerinti előzménnyel

Egy bevételezés rögzítésének lépései a gyakorlatban:

- 1. lépés — a Készletmozgás menüpont megnyitása az irányítópultról.
- 2. lépés — a bizonylatszám (pl. a beszállítói számla száma) megadása, hogy a mozgás később visszakereshető legyen.
- 3. lépés — a beérkezett termék kiválasztása a legördülő listából.
- 4. lépés — a mozgástípus kiválasztása (pl. „101 - Beszerzés”).
- 5. lépés — a bevételezett mennyiség és a beszerzési ár megadása.
- 6. lépés — a Készletmozgás mentése gombra kattintás; sikeres rögzítés esetén a rendszer visszaigazoló üzenetet jelenít meg, és a termék készlete azonnal frissül.

8.6. Készlettörténet megtekintése

Az Előzmények oldal minden bejelentkezett felhasználó számára elérhető, és a rendszerben valaha rögzített összes készletmozgást listázza, kereshető és rendezhető formában



ID	Termék	Felhasználó	Dátum	Bizonylatszám	Mennyiség	Mozgás típusa	Beszerzési ár	Eladási ár	Sorozatszám
5	Üveggolyó	Adminisztrátor	2026. 09. 13. 18:18:35	SZ-001	80	101 - Beszerzés	25 Ft	100 Ft	
4	Kis fa kocka	Adminisztrátor	2026. 09. 13. 18:18:13	SZ-001	32	101 - Beszerzés	100 Ft	300 Ft	

9. ábra - Előzmények oldal, alapértelmezett rendezéssel (dátum szerint)

8.7. Saját jelszó módosítása és kijelentkezés

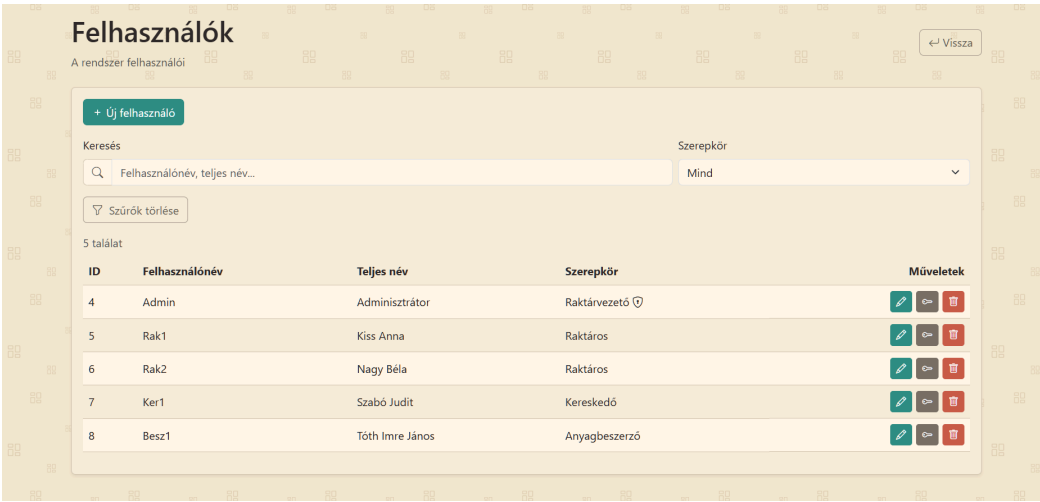
Az irányítópulton elérhető „Jelszó módosítása” gombra kattintva bármelyik bejelentkezett felhasználó módosíthatja saját jelszavát; ehhez a jelenlegi jelszó megadása is kötelező, az új jelszót pedig kétszer kell begépelni az elgépelés elkerülése végett. A módosítás után a rendszer biztonsági okból automatikusan kijelentkezteti a felhasználót, újbóli bejelentkezést kérve. A Kijelentkezés gomb bármikor lezárja a munkamenetet és törli a tárolt token.

9. Raktárvezetői (adminisztrációs) dokumentáció

A Felhasználók oldal és az azon elérhető műveletek kizárólag Raktárvezető szerepkörben érhetők el; a menüpont más szerepkörnél nem is jelenik meg az irányítópulton.

9.1. Felhasználók listázása és felvitele

A Felhasználók oldal táblázatosan listázza a rendszerben nyilvántartott összes felhasználót a nevükkel és szerepkörükkel együtt. Új felhasználó felvitelekor a Raktárvezető megadja a felhasználónevet, a teljes nevet, a kezdeti jelszót és a szerepkört; a felhasználónév egyediségét a rendszer adatbázis-szinten is kikényszeríti.



ID	Felhasználónév	Teljes név	Szerepkör	Műveletek
4	Admin	Adminisztrátor	Raktárvezető	
5	Rak1	Kiss Anna	Raktáros	
6	Rak2	Nagy Béla	Raktáros	
7	Ker1	Szabó Judit	Kereskedő	
8	Besz1	Tóth Imre János	Anyagbeszerző	

10. ábra - Felhasználók listája

9.2. Felhasználói adatok és jelszó módosítása

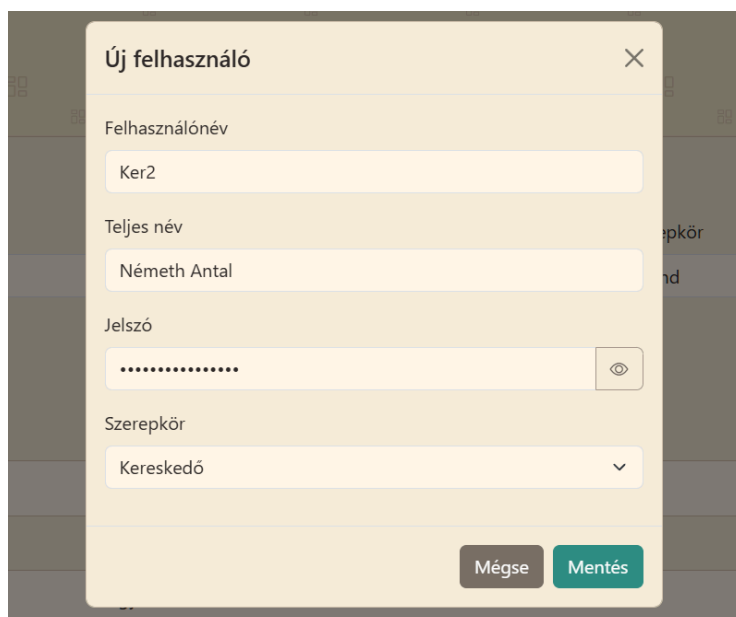
A szerkesztés ikonnal megnyitható ablakban a felhasználónév, a teljes név és a szerepkör módosítható. A jelszó módosítása külön művelet: a Raktárvezető a jelenlegi jelszó ismerete nélkül állíthat be új jelszót egy másik felhasználónak, ami tipikusan elfelejtett jelszó esetén szükséges.

9.3. Az utolsó Raktárvezető védelme

A rendszer sem szerepkör-módosítással, sem törléssel nem engedi, hogy az utolsó Raktárvezető fiók megszűnjön vagy alacsonyabb jogosultságúvá váljon — így elkerülhető, hogy a rendszer adminisztrátor nélkül maradjon. Ha a Raktárvezető ezt mégis megkísérli, a felület a backend által visszaadott hibaüzenetet jeleníti meg, és a művelet nem hajtodik végre.

9.4. Új felhasználó felvitele

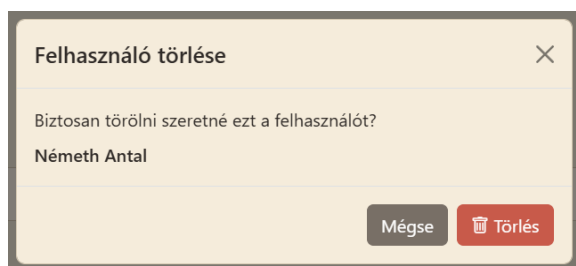
- 1. lépés — a Felhasználók oldal megnyitása az irányítópultról (csak Raktárvezetőnek jelenik meg).
- 2. lépés — az „Új felhasználó” gomb megnyomása.
- 3. lépés — a felhasználónév, a teljes név és egy kezdeti jelszó megadása.
- 4. lépés — a szerepkör kiválasztása (Raktárvezető, Raktáros, Kereskedő vagy Anyagbeszerző) a munkakörnek megfelelően.
- 5. lépés — a Mentés gomb megnyomása; már foglalt felhasználónév esetén a rendszer hibaüzenettel jelzi ezt, és a felhasználó nem jön létre.



11. ábra - Új felhasználó létrehozása ablak

9.5. Felhasználó törlése

A törlés ikon megerősítés után eltávolítja a kiválasztott felhasználót. Amennyiben a felhasználóhoz már kapcsolódik rögzített készletmozgás, a törlés az adatbázis idegenkulcs-megszorítása miatt sikertelen lesz, és a felület a megfelelő hibaüzenetet mutatja — ez szándékos védelem, amely biztosítja, hogy a korábbi mozgások mindig egy azonosítható felhasználóhoz legyenek köthetők.



12. ábra - Felhasználó törlése megerősítő ablak

10. Telepítés és üzembe helyezés

10.1. Előfeltételek

- .NET SDK (a projekt .NET 10 célkörnyezetre készült).
- Futó MySQL-kiszolgáló (XAMPP vagy WAMP is megfelelő).
- Modern böngésző.
- Visual Studio Code a Live Server kiegészítővel a frontend kiszolgálásához (vagy bármely más statikus fájlkiszolgáló).

10.2. Indítási sorrend

Lépés	Teendő
1. MySQL indítása	A MySQL-kiszolgáló elindítása (pl. XAMPP vagy WAMP vezérlőpultjáról).
2. Adatbázis előkészítése	A Database/create.sql szkript importálása phpMyAdmin-on vagy a MySQL-kliensen keresztül; ez létrehozza a warehouse adatbázist és a három táblát, előre feltöltött adatok nélkül.
3. Backend konfigurálása	Szükség esetén az appsettings.json ConnectionStrings:MySql értékének igazítása a helyi MySQL-hozzáféréshez (felhasználó, jelszó, port).
4. Backend indítása	A Backend\WarehouseAPI\WarehouseAPI mappából: dotnet run. Az API a terminálban kiírt címen indul el (fejlesztői módban a Scalar felület is elérhető).
5. Frontend kiszolgálása	A Frontend mappa megnyitása Visual Studio Code-ban, majd az index.html megnyitása a Live Server kiegészítővel (alapértelmezett cím: http://localhost:5500/Frontend/index.html).
6. Alkalmazás megnyitása	A böngészőben a Live Server által megnyitott cím felkeresése.
7. Bejelentkezés	Amennyiben az adatbázis még üres, az első (Raktárvezetői) felhasználó létrehozása a kezdőoldalon.

12. táblázat - Az alkalmazás első indítása

10.3. Gyakori induláskori hibák

Tünet	Valószínű ok és teendő
A backend indításkor hibával leáll, a Jwt:Key hosszára hivatkozva.	Az appsettings.Development.json Jwt:Key értéke hiányzik vagy 32 karakternél rövidebb; a fejlesztői fájl alapértelmezett kulcsát kell visszaállítani, vagy éles környezetben egy megfelelő hosszúságú kulcsot kell beállítani.
A frontend megnyílik, de egyetlen adat sem töltődik be.	A backend nem fut, vagy más porton fut, mint amit az api.js API_BASE_URL konstansa vár — ellenőrizni kell a backend indításakor kiírt címet.

Tünet	Valószínű ok és teendő
A böngésző konzoljában CORS-hibaüzenet jelenik meg.	A frontend nem a Cors:Origins listában szereplő címről (pl. nem az 5500-as porton) fut; a Live Server portját vagy a backend Cors:Origins beállítását kell összehangolni.
Bejelentkezés után a rendszer visszairányít a kezdőoldalra.	A JWT token lejárt vagy érvénytelen (401-es válasz); ilyenkor az authFetch automatikusan kijelentkeztet — újbóli bejelentkezés szükséges.
Az adatbázis-kapcsolat sikertelen a backend indításakor.	A MySQL-kiszolgáló nem fut, vagy a ConnectionStrings:MySql adatai (kiszolgáló, felhasználó, jelszó) nem egyeznek a helyi MySQL-beállításokkal

12.táblázat - Gyakori hibák és elhárításuk

11. Ismert korlátok, továbbfejlesztési lehetőségek

A rendszer a projekt által meghatározott követelményeket megvalósítja, a felsorolt korlátok ezért nem hibák, hanem olyan pontok, amelyek egy nagyobb léptékű, éles környezetben futó rendszerben továbbfejleszthetők lennének.

11.1. Jelenlegi korlátok

- Nincs külön automatizált unit- és integrációsteszt-rendszer; a tesztelés jelenleg manuális (lásd teszt dokumentáció).
- A frontend kliensoldalon dolgozza fel a listákat (keresés, szűrés, rendezés) lapozás nélkül, ami a jelenlegi adatmennyiség mellett megfelelő.
- A jelenlegi konfiguráció (pl. a fejlesztői JWT-kulcs) helyi fejlesztési használatra készült, éles környezetben megerősítést igényelne.

11.2. Biztonsági és üzemeltetési továbbfejlesztési irányok

A rendszer alapvető biztonsági mechanizmusai (jelszóhashelés, JWT hitelesítés, szerepkör-ellenőrzés a backendben) adottak. Éles, több felhasználós üzemeltetéshez az alábbiak lennének indokoltak:

- Titkos konfigurációk: a kapcsolati sztring és a JWT-kulcs kiszervezése környezeti változóba vagy titokkezelő szolgáltatásba, forráskódtól elválasztva.
- HTTPS: a kommunikáció titkosítása, különösen a hitelesítési adatok és a JWT token védelme érdekében.
- JWT-kezelés finomítása: refresh token, rövidebb access token élettartam és token-visszavonási lehetőség bevezetése.

11.3. Funkcionális és technikai továbbfejlesztési lehetőségek

A backend és a frontend jelenlegi felépítése jó alapot ad további fejlesztésekhez, például:

- Automatizált backend- (unit és integrációs) és frontend-tesztek bevezetése;
- Egységesebb, részletesebb API-hibaválasz-formátum és szerveroldali naplózás;
- EF Core migrációk bevezetése az EnsureCreated() helyett, valamint további adatbázis-indexek nagyobb adatmennyiség esetén;
- Szerveroldali lapozás, keresés és szűrés nagyobb terméklista esetén;
- Riportok, készletérték-statisztikák, CSV/Excel export, automatikus készletriesztás, több telephely és vonalkódolvasó-integráció.

12. Összefoglalás

A Raktárkezelő projekt egy működő, háromrétegű webalkalmazást eredményezett, amely MySQL adatbázisra, ASP.NET Core Web API backendre és keretrendszer nélküli HTML/CSS/JavaScript frontendre épül. A rendszer a raktári munka legfontosabb folyamatait fedi le: termékek nyilvántartását, négy szerepkörre bontott jogosultságkezelést, JWT alapú hitelesítést, készletmozgások (bevételezés és kiadás) rögzítését nyolc mozgástípus-kóddal, valamint a mozgások teljes körű, visszakereshető történetét.

A fejlesztés során kiemelt szempont volt, hogy a jogosultságok érvényesítése ne csak a felületen, hanem minden esetben a backendben is megtörténjen, illetve hogy a raktári folyamatokból adódó üzleti szabályok (az utolsó Raktárvezető védelme, a termékek csak 0 készlet esetén törölhetők, a készletmozgások mennyiségi és típus szerinti validációja) következetesen érvényesüljenek. A manuális funkcionális tesztelés (11. fejezet) mind a 25 vizsgált esetben megerősítette, hogy ezek a szabályok helyesen működnek.

A projekt jelenlegi terjedelmén túlmutató fejlesztési lehetőségeket a 12. fejezet foglalja össze; ezek nem a jelenlegi rendszer hiányosságai, hanem egy esetleges további fejlesztési szakasz lehetséges irányai.

13. Forrásjegyzék

A projekt fejlesztése során felhasznált technológiák hivatalos dokumentációi:

- Microsoft Learn — ASP.NET Core dokumentáció: <https://learn.microsoft.com/hu-hu/aspnet/core/?view=aspnetcore-10.0>
- Microsoft Learn — Entity Framework Core dokumentáció: <https://learn.microsoft.com/hu-hu/ef/core/>
- Microsoft Learn — JWT Bearer hitelesítés ASP.NET Core-ban: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication?view=aspnetcore-10.0>
- MySQL hivatalos dokumentáció: <https://dev.mysql.com/doc/>
- MDN Web Docs — HTML, CSS és JavaScript referencia: <https://developer.mozilla.org/en-US/>
- Bootstrap hivatalos dokumentáció: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- Bootstrap Icons: <https://icons.getbootstrap.com/>
- Scalar (OpenAPI dokumentációs felület): <https://github.com/scalar/scalar>